



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2413 - BASES DE DATOS

Solución Interrogación 1

Fecha: 8 de mayo de 2026

1^{er} semestre 2026 - Profesores: Eduardo Bustos - Christian Álvarez

Pregunta 1: Álgebra Relacional

- a) (3 pts) Nombre y nacionalidad de pilotos en el podio (PosicionFinal $\leq$ 3) en el GP de Miami.

$$\pi_{Nombre, Nacionalidad}(\sigma_{NombreGP='GPdeMiami' \wedge PosicionFinal \leq 3}(Pilotos \bowtie Resultados \bowtie Carreras))$$

- b) (3 pts) ID y nombre de pilotos que han participado en al menos una carrera, pero que nunca han obtenido una vuelta rápida.

$$\begin{aligned} & \rho(participaron, \pi_{IDP, Nombre}(Pilotos \bowtie Resultados)) \\ & \rho(con_vuelta_rapida, \pi_{IDP, Nombre}(\sigma_{VueltaRapida=1}(Pilotos \bowtie Resultados))) \\ & \quad participaron - con_vuelta_rapida \end{aligned}$$

- c) (4 pts) IDs de los pilotos que han ganado al menos 2 carreras (sin usar contadores).

$$\begin{aligned} & \rho(Ganadores, \pi_{IDP, IDC}(\sigma_{PosicionFinal=1}(Resultados))) \\ & \rho(G1, \rho_{(IDP \rightarrow IDP1, IDC \rightarrow IDC1)}(Ganadores)) \\ & \rho(G2, \rho_{(IDP \rightarrow IDP2, IDC \rightarrow IDC2)}(Ganadores)) \\ & \pi_{IDP1}(\sigma_{IDP1=IDP2 \wedge IDC1 \neq IDC2}(G1 \times G2)) \end{aligned}$$

- d) (5 pts) Nombre de escuderías cuyos pilotos terminaron en el podio en todas las carreras.

$$\rho(EscuderiasFueraPodio, \pi_{IDE}(\sigma_{PosicionFinal3}(Pilotos \bowtie Resultados)))$$

$$\rho(IDE_Exitosas, \pi_{IDE}(Escuderias) - EscuderiasFueraPodio)$$

$$\pi_{NombreE}(Escuderias \bowtie IDE_Exitosas)$$

Pregunta 2: Diagrama E/R y Esquema Relacional

a) Diagrama Entidad-Relación

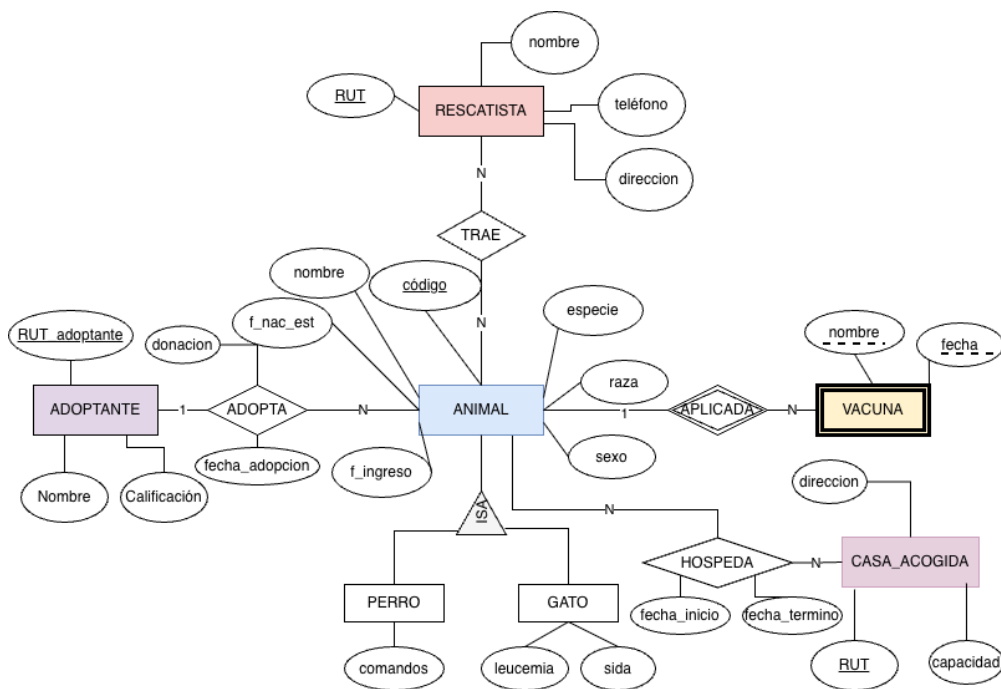


Figura 1: Diagrama Entidad Relación

b) Esquema Relacional

- **Animal** (codigo, nombre, especie, raza, sexo, fecha_nac, fecha_ing, RUT_adoptante, fecha_adopcion, donacion)
- **Perro** (codigo, comandos)
- **Gato** (codigo, leucemia, sida)
- **Vacuna** (codigo, nombre, fecha)
- **Rescatista** (RUT, nombre, telefono, direccion)
- **Traer** (RUT, codigo)

- CasaAcogida (RUT, direccion, capacidad)
- Hospeda (codigo, RUT, fecha_inicio, fecha_termino)

Pregunta 3: SQL y Lógica en la BD

a) Procedimiento sp_cancelar_vuelo

```
CREATE OR REPLACE FUNCTION sp_cancelar_vuelo(id_vuelo_param
INT) RETURNS VOID AS $$
BEGIN
    UPDATE Vuelos SET estado = 'Cancelado'
    WHERE id_vuelo = id_vuelo_param;

    UPDATE Pasajeros
    SET millas_acumuladas = millas_acumuladas +
    millas_gastadas + 500
    FROM Pasajeros
    INNER JOIN Reservas
    ON rut = rut_pasajero
    WHERE id_vuelo = id_vuelo_param
    AND millas_gastadas > 0;

    INSERT INTO AuditoriaMillas(
    rut_pasajero, operacion, cantidad)
    SELECT rut_pasajero, 'DEVOLUCION_CANCEL',
    millas_gastadas + 500
    FROM Reservas
    WHERE id_vuelo = id_vuelo_param
    AND millas_gastadas > 0;

END;
$$ LANGUAGE plpgsql;
```

b) Trigger tr_vuelo_aterrizado

```
CREATE OR REPLACE FUNCTION fn_vuelo_aterrizado()
RETURNS TRIGGER AS $$
DECLARE
    v_millas_vuelo INT;
    r RECORD;
    multiplicador FLOAT;
BEGIN
    IF (NEW.estado = 'Aterrizado') THEN
        SELECT distancia_millas INTO v_millas_vuelo FROM Vuelos WHERE
```

```

UPDATE Pasajeros
SET millas_acumuladas = millas_acumuladas + v_millas_vuelo * 2
FROM Pasajeros p
INNER JOIN
Reservas res
ON p.rut = res.rut_pasajero
AND res.id_vuelo = NEW.id_vuelo
AND res.estado_checkin = TRUE
AND p.categoria = 'Oro';

UPDATE Pasajeros
SET millas_acumuladas = millas_acumuladas + v_millas_vuelo * 1
FROM Pasajeros p
INNER JOIN
Reservas res
ON p.rut = res.rut_pasajero
AND res.id_vuelo = NEW.id_vuelo
AND res.estado_checkin = TRUE
AND p.categoria = 'Plata';

UPDATE Pasajeros
SET millas_acumuladas = millas_acumuladas + v_millas_vuelo
FROM Pasajeros p
INNER JOIN
Reservas res
ON p.rut = res.rut_pasajero
AND res.id_vuelo = NEW.id_vuelo
AND res.estado_checkin = TRUE
AND p.categoria = 'Bronce';

INSERT INTO AuditoriaMillas (rut_pasajero, operacion, cantidad)
SELECT rut_pasajero, 'ACUM_VUELO', v_millas_vuelo * 2
FROM Reservas res
WHERE res.id_vuelo = NEW.id_vuelo
and res.estado_checkin = TRUE
and p.categoria = 'Oro';

INSERT INTO AuditoriaMillas (rut_pasajero, operacion, cantidad)
SELECT rut_pasajero, 'ACUM_VUELO', v_millas_vuelo * 1.5
FROM Reservas res
WHERE res.id_vuelo = NEW.id_vuelo
and res.estado_checkin = TRUE
and p.categoria = 'Plata';

INSERT INTO AuditoriaMillas (rut_pasajero, operacion, cantidad)

```

```

        SELECT rut_pasajero, 'ACUM_VUELO', v_millas_vuelo
        FROM Reservas res
        WHERE res.id_vuelo = NEW.id_vuelo
        and res.estado_checkin = TRUE
        and p.categoria = 'Bronce';

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_vuelo_aterrizado
AFTER UPDATE ON Vuelos
FOR EACH ROW
EXECUTE FUNCTION fn_vuelo_aterrizado();

```

c) (5 pts) Trigger de restricción de Check-in.

```

CREATE OR REPLACE FUNCTION fn_impedir_checkin()
RETURNS TRIGGER AS $$
DECLARE
    v_estado VARCHAR;
BEGIN
    SELECT estado INTO v_estado
    FROM Vuelos
    WHERE id_vuelo = NEW.id_vuelo;

    IF (NEW.estado_checkin = TRUE
    AND v_estado != 'Programado') THEN
        RAISE EXCEPTION 'No se puede realizar
        check-in: el vuelo no est  Programado';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_checkin_restriccion
BEFORE INSERT OR UPDATE ON Reservas
FOR EACH ROW EXECUTE FUNCTION fn_impedir_checkin();

```

Pregunta 4: Formas Normales

a) Clave Primaria y DFs

- **PK:** $\{ID_Vuelo, ID_Emp\}$.
- **DFs:**
 - $ID_Emp \rightarrow \{Nombre_Emp, Rol\}$
 - $ID_Vuelo \rightarrow \{Avion_Reg, Modelo, Capacidad\}$
 - $Avion_Reg \rightarrow Modelo$
 - $Modelo \rightarrow Capacidad$

b) Incumplimiento 2NF

Debido a dependencias parciales ($ID_Emp \rightarrow Nombre_Emp$),
($ID_Vuelo \rightarrow Avion_Reg$)

c) Descomposición 3NF

- Empleados(ID_Emp, Nombre, Rol)
- Modelos(Modelo, Capacidad)
- Aviones(AvionReg, Modelo)
- Vuelos(IDVuelo, AvionReg)
- Asignaciones(IDVuelo, IDEmp).

d) ¿Está en BCNF?. Sí, ya que en todas las tablas resultantes, para cada dependencia funcional $X \rightarrow Y$, X es una superclave.

Bonus

- a) Conflict Serializable: Un itinerario es conflict serializable si es equivalente a algún itinerario serial.
- b) Diferencia: La Atomicidad asegura que la transacción se realice completa o no se realice nada ("todo o nada"); la Consistencia asegura que la base de datos pase de un estado válido a otro estado válido respetando las reglas de integridad.
- c) Sí: Un itinerario serial no presenta conflictos de lectura/escritura entrelazados porque las transacciones se ejecutan una después de la otra sin solapamiento.
- d) Sí: Un itinerario serial no presenta conflictos de lectura/escritura entrelazados porque las transacciones se ejecutan una después de la otra sin solapamiento.

- e) Long Transaction: Es una transacción que permanece activa por un periodo prolongado de tiempo. Afecta al DBMS porque mantiene bloqueos (locks) durante mucho tiempo, degradando el rendimiento y aumentando la probabilidad de deadlocks.
- f) Deadlock: Es una situación donde dos o más transacciones se bloquean mutuamente esperando recursos que la otra posee. Afecta al sistema paralizando dichas transacciones hasta que el DBMS detecta el ciclo y aborta una de ellas.